TECHNICAL REFERENCE & CURRICULUM GUIDE

Module 10 (Part 2): Fine-Tuning, PEFT, Alignment & Distributed Training

Module 10 Part 2 breaks down the mechanics of Parameter-Efficient Fine-Tuning (PEFT) and post-SFT alignment: hyperparameter selection heuristics, 4-bit quantization dynamics (QLoRA), memory budgeting, end-to-end training implementation, and preference optimization (RLHF/PPO vs. DPO).

10.8 Choosing Rank (r), Alpha (α), and Target Modules for a Given Task & Model Size

Selecting LoRA hyperparameters requires balancing **representational capacity** against **trainable parameter volume** and **overfitting risk**.

```
                LORA HYPERPARAMETER TUNING SPECTRUM
┌──────────────────────────────────┬──────────────────────────────────┐
│     CONSERVATIVE / FORMATTING    │    COMPLEX REASONING / DOMAIN    │
├──────────────────────────────────┼──────────────────────────────────┤
│ Rank:  r ∈ [8, 16]               │ Rank:  r ∈ [32, 64]              │
│ Alpha: α = 16 or 32 (α = 2r)     │ Alpha: α = 64 or 128 (α = 2r)    │
│ Targets: Attention only          │ Targets: All Linear Modules      │
│ (q_proj, v_proj)                 │ (q, k, v, o, gate, up, down)     │
│                                  │                                  │
│ Use Case: Classification, tone,  │ Use Case: Math, code synthesis,  │
│ JSON extraction, simple chat.    │ new language adaptation.         │
└──────────────────────────────────┴──────────────────────────────────┘
```

1. Intrinsic Rank (r) Selection Rules

- ***r* ∈ *[8, 16]* (Format, Style, & Classification):** When the underlying base model already possesses domain knowledge and only needs behavioral steering (e.g., matching a strict JSON schema, adopting a persona, or text classification), rank 8 or 16 is sufficient. Higher rank risks overfitting to syntax boilerplate.
- ***r* ∈ *[32, 64]* (Reasoning, Code Generation, & Mathematical Logic):** Tasks requiring multi-step symbolic derivation or novel API tool syntax demand higher rank capacity to represent complex weight deltas (ΔW).
- **Diminishing Returns ($r > 64$):** Empirical findings (Hu et al., Dettmers et al.) demonstrate that scaling r beyond 64 yields marginal perplexity gains while linearly increasing optimizer state VRAM and adapter weight sizes. Full fine-tuning parity is typically reached at $r = 64$ when applied across all linear layers.

2. LoRA Alpha (α) and the Scaling Multiplier

Recall that LoRA scales the low-rank delta before adding it to the base activation:

$$\Delta W \cdot x = \frac{\alpha}{r} (B \cdot A) x$$

- α acts as a **structural learning rate multiplier**.
- **The Rule of Thumb ($\alpha = 2 \times r$ or $\alpha = r$):** Setting $\alpha = 2r$ makes the scaling factor $\alpha / r = 2.0$, slightly prioritizing the adapter's initial gradient updates relative to the frozen base weights.
- **Why Alpha Matters When Tuning Rank:** If you experiment with different values of r (e.g., testing $r = 16$, then $r = 32$, then $r = 64$), keeping the ratio α / r constant (e.g., fixed at 2.0) ensures the scale of the adapter updates remains invariant, meaning you do not need to re-tune your optimizer learning rate (η) for every rank change.

3. Target Module Selection: Attention vs. All Linear

In modern Transformers (LLaMA, Mistral, Qwen), a standard Decoder layer contains 7 distinct linear projections:

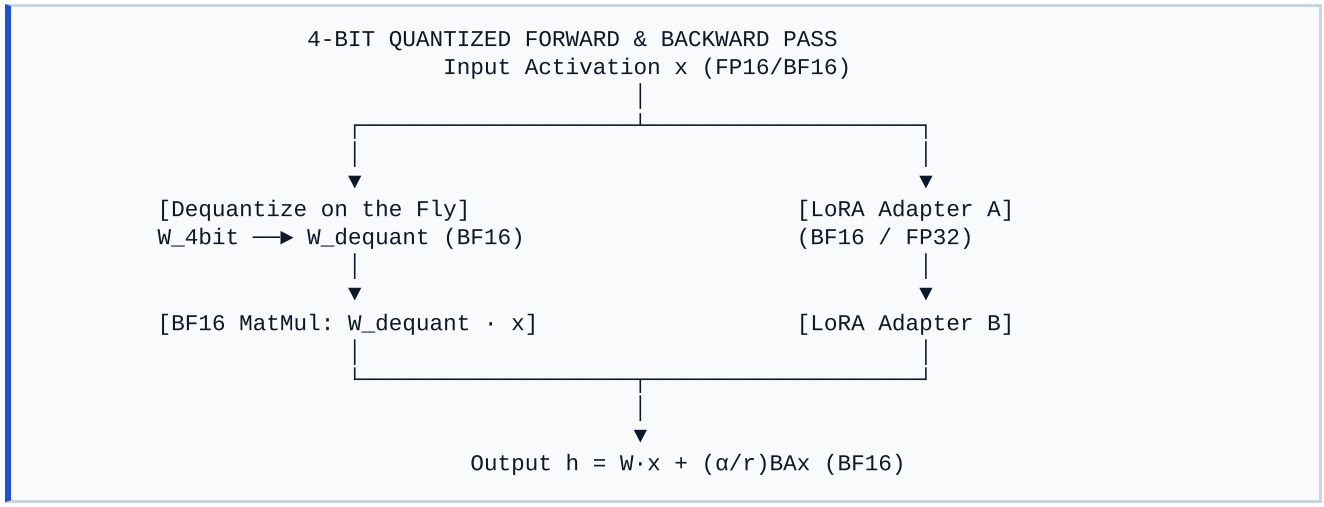
1. **Self-Attention Sub-Block:** `q_proj`, `k_proj`, `v_proj`, `o_proj`
2. **Feed-Forward / SwiGLU Sub-Block:** `gate_proj`, `up_proj`, `down_proj`

Production Target Module Guidelines

Configuration	Target Projections	Trainable Param %	Training Dynamics & Expressivity
Minimal / Legacy	`["q_proj", "v_proj"]`	~0.05% – 0.1%	Fast, low VRAM; struggles on complex multi-turn reasoning and math.
Full Attention	`["q_proj", "k_proj", "v_proj", "o_proj"]`	~0.2% – 0.4%	Good for instruction-following and task steering; captures attention routing.
All Linear (Production Default)	`["q_proj", "k_proj", "v_proj", "o_proj", "gate_proj", "up_proj", "down_proj"]`	**~1.0% – 2.5%**	**Optimal performance.** Closely matches full fine-tuning. FFN layers store factual and associative domain knowledge.

10.9 Quantization Primer: Why 4-Bit Training Is Mathematically Possible

During traditional full fine-tuning, weights, activations, and gradients must remain in high precision (FP32 or FP16) to avoid gradient underflow. Understanding why **4-bit quantized base weights can still participate in gradient descent** requires separating the forward inference pass from backpropagation.



1. Static Storage vs. Dynamic Computation

In a LoRA setup, the base weights W_O are **completely frozen**—they receive zero gradient updates:

$$\frac{\partial L}{\partial W_0} = \mathbf{0} \quad \text{(Never computed)}$$

- Because W_O never receives gradients, it does not require an optimizer state (saving 12 bytes/param) and does not require gradient storage (saving 2 bytes/param).
- The base weights can be compressed down to **4 bits (0.5 bytes per parameter)** for storage in GPU VRAM.

2. On-the-Fly Dequantization

A GPU cannot directly multiply a 4-bit integer with a 16-bit floating-point activation tensor.

1. The 4-bit base weight tensor resides in GPU High-Bandwidth Memory (HBM).
2. During the forward pass, small blocks of 4-bit weights are loaded into the streaming multiprocessor's fast **on-chip SRAM**.
3. Specialized CUDA kernels dequantize the 4-bit values into 16-bit Brain Floating Point (**BF16**) numbers *on the fly*.
4. The standard matrix multiplication ($W_{dequant} \cdot x$) executes in full BF16 precision.
5. The intermediate BF16 weights are immediately cleared from SRAM; only the compact 4-bit weights remain in global VRAM.

3. Gradient Flow During Backpropagation

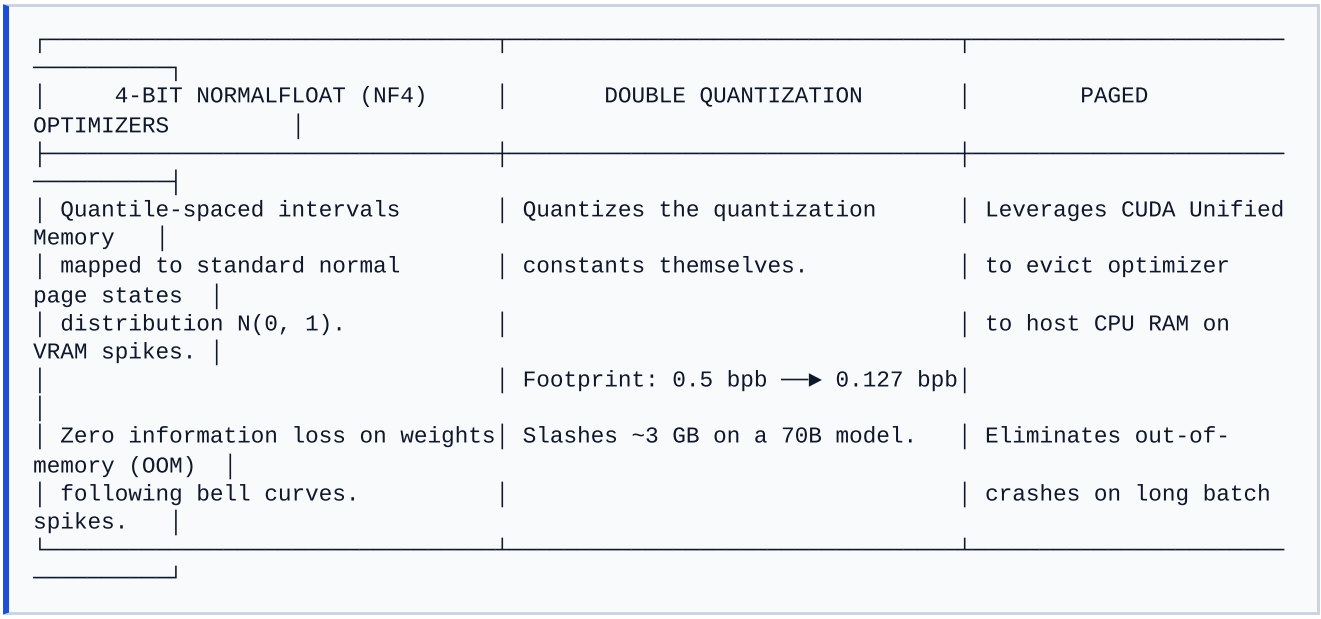
When backpropagating from the loss function L:

$$\frac{\partial L}{\partial x} = \left(W_{\text{dequant}} + \frac{\alpha}{r} B A \right)^T \frac{\partial L}{\partial h}, \quad \frac{\partial L}{\partial A} = \frac{\alpha}{r} B^T \left(\frac{\partial L}{\partial h} \right) x^T, \quad \frac{\partial L}{\partial B} = \frac{\alpha}{r} \left(\frac{\partial L}{\partial h} \right) (A x)^T$$

- Gradients flow **through** the dequantized base weights to calculate the activation gradient $\partial L / \partial x$ for upstream layers.
- Gradients are **only accumulated and optimized for matrices A and B**, which are maintained in standard 16-bit or 32-bit precision.

10.10 QLoRA Mechanics: NF4, Double Quantization, and Paged Optimizers

Introduced by Dettmers et al. (NeurIPS 2023), **QLoRA** introduces three algorithmic innovations that compress base models to 4 bits with near-zero degradation in downstream fine-tuning performance.



1. 4-bit NormalFloat (NF4) Quantization

Standard uniform quantization divides the range *[min, max]* into 2^k equally spaced bins. However, pre-trained neural network weights are not uniformly distributed; they follow a zero-centered **Normal Gaussian distribution** $\mathcal{N}(0, \sigma^2)$. Uniform bins waste precision on the sparse tails while over-saturating the dense center.

NF4 creates an **information-theoretically optimal quantile distribution**:

1. It calculates the quantiles q_i of a theoretical standard normal distribution $\mathcal{N}(0, 1)$ such that every bin has an **equal probability mass of 2^{-k}**:

$$\int_{q_i}^{q_{i+1}} \frac{1}{\sqrt{(2\pi)}} e^{-x^2/2}\, dx = \frac{1}{2^k} = \frac{1}{16}$$

2. The 16 discrete values are normalized into the symmetric interval $[-1.0, 1.0]$.
3. For any weight block W, the tensor is normalized by its absolute maximum $|W|_{max}$, and each weight coordinate is rounded to the closest of the 16 precomputed NF4 values:

$$W_{\text{NF4}} = \arg\min_{c_i} | (W / |W|_{\max}) - c_i |, \quad c_i \in \text{NF4 Table}$$

2. Double Quantization (DQ)

In block-wise quantization, weights are grouped into small blocks (e.g., block size $B = 64$) to isolate outliers. Each block requires an FP32 quantization constant (scale factor c_1).

- Storing an FP32 (32-bit) constant for every 64 parameters adds:

  $$32 \text{ bits} / 64 \text{ parameters} = 0.5 \text{ bits per parameter (bpb)}$$

  This constitutes a massive 12.5% memory overhead on top of 4-bit weights!
- **Double Quantization** treats the scale factors c_1 as inputs to a second quantization stage:
  - It groups the c_1 constants into blocks of 256.
  - It quantizes these constants into 8-bit integers (FP8) with an FP32 second-stage scale constant c_2.
  - **Memory Footprint Slashed:**

    $$(8 \text{ bits} / 64) + (32 \text{ bits} / (64 \times 256)) = 0.125 + 0.00195 = \mathbf{0.127 \text{ bpb}}$$

  - Double quantization saves ≈ 0.373 bits per parameter, freeing up **~3 GB of VRAM** on a 70B parameter model.

3. Paged Optimizers

During long-context training, activation memory fluctuates based on sequence length. A sudden batch containing long sequences can cause an instantaneous VRAM spike, triggering a fatal CUDA Out-of-Memory (OOM) crash.

- **Mechanics:** Paged Optimizers allocate memory for the non-critical optimizer states (the LoRA adapter gradients and Adam momentum states) using **CUDA Unified Memory**.
- When GPU VRAM approaches capacity, the NVIDIA driver automatically page-swaps non-active optimizer state memory blocks across PCIe lanes into standard CPU host RAM.

- Once the forward-backward pass finishes and gradient updates are calculated, the pages are paged back into GPU VRAM. This prevents intermittent OOM crashes with negligible throughput overhead.

10.11 Memory Math: Estimating VRAM Needs for Fine-Tuning at 7B, 13B, and 70B

A critical engineering task before spinning up cloud GPU clusters is calculating exact memory consumption.

1. The Unified Fine-Tuning VRAM Equation

$$\text{Total VRAM} = M_{\text{weights}} + M_{\text{opt_lora}} + M_{\text{grad_lora}} + M_{\text{activations}} + M_{\text{cuda_overhead}}$$

1. **Quantized Base Weight Memory ($M_{weights}$):**

   $$M_{\text{weights}} = N_{\text{params}} \times 0.5 \text{ bytes (in 4-bit QLoRA with DQ)}$$

2. **LoRA Adapter Weights & Gradients ($M_{opt_lora} + M_{grad_lora}$):**
   - LoRA parameters (N_{lora}) typically represent 1% of base parameters ($0.01 \times N$).
   - Adapter weights: $N_{lora} \times 2$ bytes (BF16).
   - Adapter gradients: $N_{lora} \times 2$ bytes (BF16).
   - AdamW optimizer states: $N_{lora} \times 12$ bytes (FP32).
   - Total Adapter Footprint: $N_{lora} \times (2 + 2 + 12) = N_{lora} \times 16$ bytes.
3. **Activation Memory ($M_{activations}$):** Depends on batch size B, sequence context length S, number of layers L, and hidden dimension d_{model}. When using **Activation Checkpointing (Gradient Checkpointing)**, intermediate activations are discarded during the forward pass and recomputed during the backward pass, reducing activation memory from $O(L)$ to $O(1)$ per layer:

   $$M_{\text{activations}} \approx 2 \times B \times S \times d_{\text{model}} \times n_{\text{heads}} \times \text{layers}$$

   With activation checkpointing enabled, activation memory generally consumes **1.5 GB to 4 GB** for batch size 1 to 2 at 2048 sequence length.
4. **CUDA Runtime & Framework Context Overhead ($M_{cuda_overhead}$):** PyTorch context, CUDA kernels, and memory allocator fragmentation consume **~1.5 GB to 2.5 GB**.

2. Comprehensive Production VRAM Estimator Table

Model Size	Precision & Method	Base Weights VRAM	LoRA & Opt VRAM (r=16)	Activations (Seq=2048, B=1)	CUDA Context	Total VRAM	Target GPU Tier
7B / 8B	Full Fine-Tuning (FP16)	16.0 GB	112.0 GB (Full AdamW)	6.0 GB	2.5 GB	**~136.5 GB**	2x A100 / H100 (80GB)
7B / 8B	Standard LoRA (FP16)	16.0 GB	1.2 GB	2.5 GB	1.5 GB	**~21.2 GB**	1x RTX 3090 / 4090 (24GB)
7B / 8B	**QLoRA (NF4 4-bit)**	**4.8 GB**	**1.2 GB**	**2.0 GB**	**1.2 GB**	**~9.2 GB**	**1x RTX 4060 Ti / T4 (16GB)**
13B / 14B	Standard LoRA (FP16)	28.0 GB	2.2 GB	3.5 GB	1.8 GB	**~35.5 GB**	1x A100 (40GB/80GB)
13B / 14B	**QLoRA (NF4 4-bit)**	**8.5 GB**	**2.2 GB**	**2.8 GB**	**1.5 GB**	**~15.0 GB**	**1x RTX 3090 / 4090 (24GB)**
70B	Standard LoRA (FP16)	140.0 GB	11.2 GB	14.0 GB	3.0 GB	**~168.2 GB**	4x A100 (80GB)
70B	**QLoRA (NF4 4-bit)**	**41.0 GB**	**11.2 GB**	**8.5 GB**	**2.5 GB**	**~63.2 GB**	**1x A100 / H100 (80GB)**

10.12 Single-GPU Fine-Tuning Workflow End-to-End

Below is a complete, runnable script implementing a production QLoRA fine-tuning pipeline on a single GPU (e.g., an RTX 3090, RTX 4090, or single A100) using **Hugging Face `transformers`**, **`peft`**, **`bitsandbytes`**, and **`trl` (SFTTrainer)**.

```
import torch
from datasets import Dataset
from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    BitsAndBytesConfig,
    TrainingArguments
)
from peft import LoraConfig, get_peft_model, prepare_model_for_kbit_training
from trl import SFTTrainer

# ==============================================================================
# 1. QUANTIZATION CONFIGURATION (NF4 + DOUBLE QUANTIZATION)
# ==============================================================================

bnb_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",               # 4-bit NormalFloat
    bnb_4bit_use_double_quant=True,          # Quantize quantization constants
    bnb_4bit_compute_dtype=torch.bfloat16    # Dequantize to BF16 for linear compute
)

model_id = "meta-llama/Meta-Llama-3-8B-Instruct"

tokenizer = AutoTokenizer.from_pretrained(model_id)
tokenizer.pad_token = tokenizer.eos_token     # Ensure padding token exists
tokenizer.padding_side = "right"              # Prevents causal attention alignment bugs

# Load base model directly into 4-bit VRAM footprint (~5.5 GB)
model = AutoModelForCausalLM.from_pretrained(
    model_id,
    quantization_config=bnb_config,
    device_map="auto",
    torch_dtype=torch.bfloat16
)

# Prepare model layers for k-bit training (casts norms to FP32, enables gradient
checkpoints)
model = prepare_model_for_kbit_training(model)

# ==============================================================================
# 2. LORA ADAPTER CONFIGURATION (TARGETING ALL LINEAR LAYERS)
# ==============================================================================

peft_config = LoraConfig(
    r=16,                                    # Intrinsic rank
    lora_alpha=32,                           # Alpha scalar (alpha = 2r)
    lora_dropout=0.05,                       # Regularization dropout on adapter outputs
    bias="none",
    task_type="CAUSAL_LM",
    target_modules=[                         # All linear modules in LLaMA architecture
        "q_proj", "k_proj", "v_proj", "o_proj",
        "gate_proj", "up_proj", "down_proj"
    ]
)

model = get_peft_model(model, peft_config)
model.print_trainable_parameters()
# Output: trainable params: ~20M || all params: ~8B || trainable%: ~0.25%

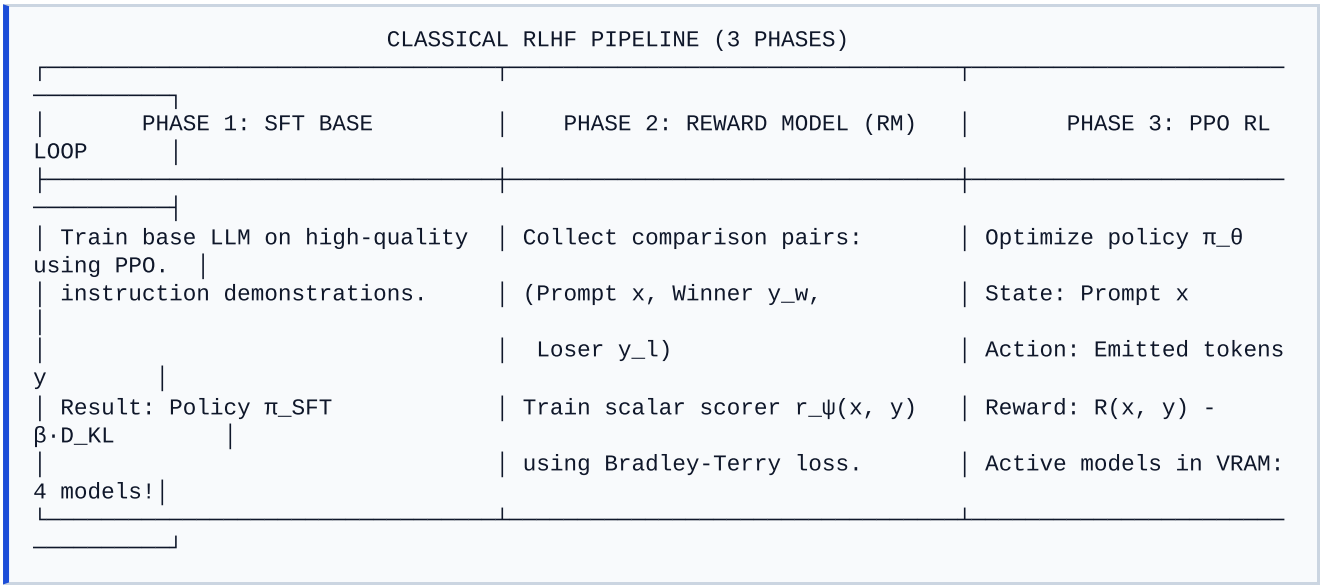
# ==============================================================================
# 3. SAMPLE INSTRUCTION DATASET
# ==============================================================================

sample_data = [
    {
```

10.13 Alignment Beyond SFT: The Classical RLHF Pipeline (Reward Modeling + PPO)

Supervised Fine-Tuning (SFT) teaches a model **how to structure answers and follow instructions**, but it suffers from the limitation of cross-entropy loss: it treats all ground-truth tokens equally. It does not teach the model to distinguish between a mediocre response and an exceptional one, nor does it penalize toxic, misleading, or sycophantic behavior.

Reinforcement Learning from Human Feedback (RLHF) (Christiano et al., 2017; Ouyang et al., 2022) aligns the model with human preferences across helpfulness, truthfulness, and safety.



Phase 1: Supervised Fine-Tuning (Policy Initialization)

Train the foundational base model on high-quality instruction datasets to produce the baseline policy model π^{SFT}.

Phase 2: Reward Model Training (Bradley-Terry Preference Modeling)

1. Sample prompts x from a dataset.
2. Generate pairs of completions from the SFT model: $y_1, y_2 \sim \pi^{SFT}(\cdot \mid x)$.
3. Human evaluators (or an AI judge) rank the completions, designating a preferred **winner** y_w and a **loser** y_l ($y_w \succ y_l \mid x$).
4. A **Reward Model** $r_\psi(x, y)$ (typically initialized from the SFT checkpoint with the final LM classification head replaced by a scalar projection head $\mathbb{R}$) is trained to output a scalar reward score.

5. **Bradley-Terry Loss Formulation:** The model minimizes the negative log-likelihood of the human preference distribution:

$$L_{\text{RM}}(\psi) = -\mathbb{E}_{(x, y_w, y_l)} \left[\log \sigma(r_\psi(x, y_w) - r_\psi(x, y_l)) \right]$$

where $\sigma(z) = 1 / (1 + e^{-z})$ is the sigmoid function. Minimizing this loss maximizes the reward score margin between y_w and y_l.

Phase 3: Proximal Policy Optimization (PPO)

In Phase 3, the SFT model acts as an agent whose "actions" are the tokens it autoregressively generates.

- **The Total Reward Function with KL Penalty:** Left unconstrained, the policy network would quickly exploit vulnerabilities in the reward model—emitting repetitive, non-sensical, or hacky responses that receive high reward scores (**Reward Hacking**).
- To prevent policy drift, a **Kullback-Leibler (KL) Divergence penalty** is subtracted from the reward:

$$R_{\text{total}}(x, y) = r_\psi(x, y) - \beta D_{\text{KL}}(\pi_\theta(y \mid x) \,\|\, \pi^{\text{SFT}}(y \mid x))$$

where β is a dynamic hyperparameter controlling the strength of the anchor to the original SFT model.

- The policy is updated using the **PPO Clipped Objective**:

$$L_{\text{PPO}}(\theta) = -\mathbb{E}_t \left[\min(\rho_t(\theta) \hat{A}_t , \text{clip}(\rho_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t) \right]$$

where $\rho_t(\theta) = \pi_\theta(a_t \mid s_t) / \pi_{\theta_{old}}(a_t \mid s_t)$ is the probability ratio, and $\hat{A}_t$ is the Generalized Advantage Estimator (GAE).

The Production Bottleneck of PPO

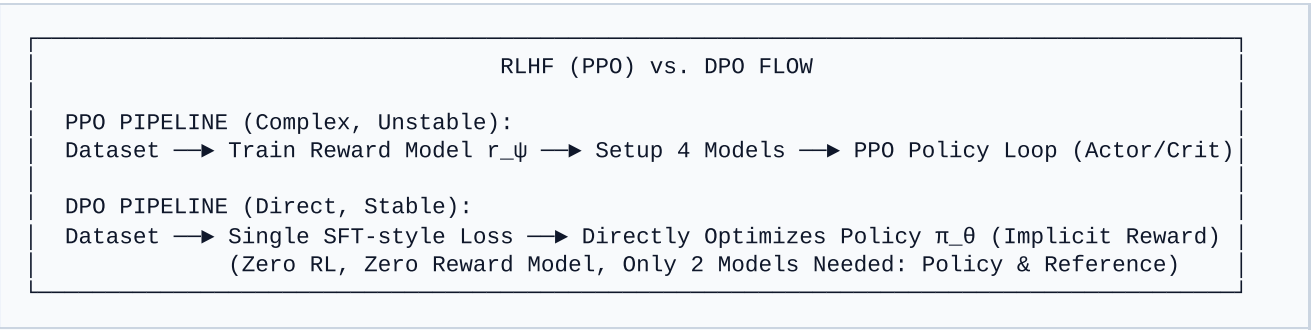
Running Phase 3 requires holding **four distinct models concurrently in memory**:

1. **Actor Network (π_θ):** The active policy being trained.
2. **Critic Network (V_ϕ):** The value function estimating expected future returns.
3. **Reference Model (π^{SFT}):** The frozen SFT model used to calculate the KL divergence penalty.
4. **Reward Model (r_ψ):** The frozen scalar scoring network.

This infrastructure overhead makes PPO complex, computationally expensive, and notoriously hyperparameter-sensitive to train.

10.14 Direct Preference Optimization (DPO): The Stable Alternative to PPO

Introduced by Rafailov et al. (Stanford, NeurIPS 2023), **Direct Preference Optimization (DPO)** revolutionizes alignment by demonstrating that **the constrained RLHF objective can be solved exactly without ever training a separate reward model or using reinforcement learning**.



1. The Mathematical Derivation of DPO

Rafailov et al. started from the exact mathematical formulation of the KL-constrained RLHF optimization objective:

$$\max_{\pi} \mathbb{E}_{x \sim \mathcal{D},\, y \sim \pi} [\, r(x, y) \,] - \beta\, D_{\mathrm{KL}}(\, \pi(y \mid x) \,\|\, \pi_{\mathrm{ref}}(y \mid x) \,)$$

Through mathematical rearrangement, the optimal policy π^* under this objective can be expressed analytically in closed form:

$$\pi^*(y \mid x) = \frac{1}{Z(x)}\ \pi_{\mathrm{ref}}(y \mid x) \exp\left(\ \frac{1}{\beta}\ r(x, y)\ \right)$$

where $Z(x) = \sum_y \pi_{ref}(y \mid x) \exp((1 / \beta)\, r(x, y))$ is the partition function.

By taking the natural logarithm of both sides and isolating the reward term $r(x, y)$, they revealed that **the ground-truth reward can be expressed directly in terms of the optimal policy and reference policy**:

$$r(x, y) = \beta \log \frac{\pi^*(y \mid x)}{\pi_{\mathrm{ref}}(y \mid x)} + \beta \log Z(x)$$

2. The DPO Objective Loss Function

When this expression for $r(x, y)$ is substituted directly into the Bradley-Terry preference model (from Phase 2 of RLHF), the unknown partition function $Z(x)$ cancels out entirely:

$$L_{\mathrm{DPO}}(\theta\,;\, \pi_{\mathrm{ref}}) = -\mathbb{E}_{(x,\, y_w,\, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\mathrm{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\mathrm{ref}}(y_l \mid x)} \right) \right]$$

3. Gradient Dynamics: How DPO Works

The gradient of the DPO loss with respect to parameters θ has an intuitive interpretation:

$$\nabla_\theta L_{\text{DPO}}(\theta) = -\beta\, \sigma(\, \hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w)\,)\, [\, \nabla_\theta \log \pi_\theta(y_w \mid x) - \nabla_\theta \log \pi_\theta(y_l \mid x)\,]$$

- **The Direction:** It increases the likelihood of preferred tokens in y_w ($\nabla_\theta \log \pi_\theta(y_w \mid x)$) while actively decreasing the likelihood of rejected tokens in y_l ($-\nabla_\theta \log \pi_\theta(y_l \mid x)$).
- **The Dynamic Weight:** The update is weighted by $\sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w))$—the error of the implicit reward model. When the policy already ranks y_w much higher than y_l, the weight scales to zero, preventing over-optimization. If the model incorrectly prefers y_l, the gradient pushes back aggressively.

4. PPO vs. DPO Comparison

Dimension	PPO (Classical RLHF)	DPO (Direct Preference Optimization)
Underlying Math	Reinforcement Learning (Actor-Critic / Policy Gradient)	Direct Supervised Classification (Implicit Reward Closed Form)
Active Models in Memory	**4 models** (Actor, Critic, Reference, Reward)	**2 models** (Active Policy π_θ, Frozen Reference π_{ref})
Training Stability	Highly unstable; prone to reward collapse & divergence	**Exceptionally stable**; standard cross-entropy-style training
Compute Overhead	Massive; high GPU cluster requirement	Low; fits on standard single-node or single-GPU setups
Sample Efficiency	Requires online dynamic rollout sampling during training	Works directly on offline static (x, y_w, y_l) datasets

5. Production DPO Implementation (TRL Library)

```
import torch
from datasets import Dataset
from transformers import AutoModelForCausalLM, AutoTokenizer
from trl import DPOTrainer, DPOConfig
from peft import LoraConfig

# Load models (Reference model can share base weights if using LoRA!)
model_id = "meta-llama/Meta-Llama-3-8B-Instruct"
tokenizer = AutoTokenizer.from_pretrained(model_id)
tokenizer.pad_token = tokenizer.eos_token

# DPO Dataset Format: prompt, chosen (winner), rejected (loser)
dpo_data = [
    {
        "prompt": "How should I structure an API response for an authentication failure?",
        "chosen": "Return HTTP 401 Unauthorized with a structured JSON payload: {\"error\":
\"invalid_credentials\", \"message\": \"Authentication failed.\"}.",
        "rejected": "Just print an error message saying login failed and redirect back to
the home page."
    }
]
train_dataset = Dataset.from_list(dpo_data)

# When using LoRA with DPOTrainer, the reference model is handled implicitly
# by freezing the base model and disabling adapter forward passes!
peft_config = LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=["q_proj", "v_proj"],
    task_type="CAUSAL_LM"
)

dpo_config = DPOConfig(
    output_dir="./dpo_aligned_model",
    beta=0.1,                                   # KL penalty coefficient (typically 0.05 to
0.5)
    learning_rate=5e-6,                         # DPO uses low learning rates to prevent drift
    per_device_train_batch_size=1,
    gradient_accumulation_steps=4,
    max_length=1024,
    max_prompt_length=512,
    bf16=True,
    logging_steps=10
)

# Initialize DPO Trainer
dpo_trainer = DPOTrainer(
    model=model_id,                             # Active policy model
    ref_model=None,                             # Handled automatically via LoRA adapter
freezing
    args=dpo_config,
    train_dataset=train_dataset,
    tokenizer=tokenizer,
    peft_config=peft_config
)

# Execute DPO Alignment
print("Starting Direct Preference Optimization (DPO)...")
# dpo_trainer.train()
```

Technical Summary of Module 10 Part 2

Topic	Primary Mechanism	Critical Formula / Invariant	Failure Mode Addressed
10.8 LoRA Hyperparameters	Rank r capacity vs. Target modules	$\alpha = 2r$; Target all linear layers (`q, k, v, o, gate, up, down`)	Under-parameterized adapters failing on reasoning tasks
10.9 Quantization Basics	Static 4-bit storage + Dynamic BF16 compute	W_o frozen ($\nabla_{W_o} = \boldsymbol{o}$); Dequantized in SRAM on the fly	High VRAM memory footprint during training
10.10 QLoRA Mechanics	NF4 quantiles + Double Quantization + Paged Optimizers	Footprint reduced to 0.127 bpb scale overhead; Unified memory paging	Out-of-memory crashes on activation spikes
10.11 Memory Math	Unified VRAM estimator equation	$\text{Static VRAM} \approx 0.5$ bytes/param + LoRA overhead	GPU cluster misconfiguration & out-of-memory crashes
10.12 Single-GPU Workflow	End-to-end `transformers` + `peft` + `bitsandbytes` + `trl`	SFTTrainer with `prepare_model_for_kbit_training()`	Disjointed fine-tuning tooling & script crashes
10.13 Classical RLHF	Reward Model + PPO policy updates	$R(x, y) = r_\psi(x, y) - \beta D_{KL}(\pi_\theta \| \pi^{SFT})$	Policy collapse, reward hacking, & 4-model cluster VRAM bloat
10.14 DPO Alignment	Closed-form implicit reward substitution	$L_{DPO} = -\log \sigma(\beta \log (\pi(y_w)/\pi_{ref}(y_w)) - \beta \log (\pi(y_l)/\pi_{ref}(y_l)))$	PPO instability & complex reward model pipelines